

spread across many servers. In general, external storage could mean any storage system that is slower to access than main memory and addressable by some integer. The file systems used on external storage devices can be quite complex. For now, just think of them as a very large storage space.

In this section we begin by describing various aspects of external data handling. We talk about a simple approach to organizing external data: sequential ordering. Finally, we discuss B-trees and explain why they work so well with disk files. We finish with another approach to external storage, indexing, which can be used alone or with a B-tree. We also touch on other aspects of external storage, such as searching techniques.

The details of external storage techniques are dependent on the operating system, language, and even the hardware used in a particular installation. Here, we stick with a general discussion to get the concepts across.

Accessing External Data

The data structures discussed so far are all based on the assumption that data is stored entirely in memory (sometimes called main memory or *RAM*, for random-access memory). Every element in RAM is accessible in the same amount of time if you have its address. In many situations, however, the amount of data to be processed is too large to fit in memory all at once. In this case a different kind of storage is necessary. External storage generally has a much larger capacity than main memory—made possible by a lower cost per byte of storage.

Of course, disk files have another advantage over most RAM: their permanence. When you turn off your computer (or the power fails), the data in main memory is lost. Disk files and other **nonvolatile data storage** devices can retain data indefinitely with the power off. RAM is a volatile data store because its contents can be easily lost.

The disadvantage of external storage is its speed; it's *much slower* than main memory. This speed difference means that different techniques must be used to handle it efficiently. All-important data must be kept on some kind of device where it won't be lost. Moving the data efficiently between slower permanent stores to faster volatile stores like RAM motivates the need for different kinds of data structures.

As an example of external storage, imagine that you're writing a database program to handle the basic contact data for everyone living in a particular state or country—perhaps 1 to 100 million entries. Each entry includes a surname, given name, phone number, address, and various other data. Let's say an entry is stored as a record requiring 1,024 bytes, a kilobyte. The result is a database size of at least $1,000,000 \times 1,024$, which is 1,024,000,000 bytes, or close to 1 gigabyte. There was a day when no computer on earth had a gigabyte of memory. Now gigabyte storage is commonplace, and machines can have terabytes of memory. We'll assume that there is some collection of data that is too large to fit in main memory but small enough to fit on the disk drive of some target machine. This could be for a larger collection of data, like all the credit card transactions in the world for a year.

Thus, we have a large amount of data on a disk or other slow drive. How do you organize it to provide the usual desirable characteristics of data structures: quick search, insertion, and deletion?

In investigating the answers, keep in mind two constraints. First, accessing external data is much slower than accessing it in main memory. Second, external data access typically reads or writes many records at once. Let's explore these points.

Very Slow Access

A computer's main memory works electronically. Any byte can be accessed just as fast as any other byte. The access time depends on the technology being used, but let's assume it takes something like 10 nanoseconds (10 billionths of a second). How much slower does it take to get a byte from external storage?

Let's first consider disk drives. Data is arranged in circular tracks on a spinning disk. To access a particular piece of data on a disk drive, the read-write head must first be moved to the correct track. This is done with a stepping motor or similar device; it's a mechanical activity that requires several milliseconds (thousandths of a second).

After the correct track is found, the read-write head must wait for the data to rotate into position. On average, it takes half a revolution for the head to be over the data. Even if the disk is spinning at 15,000 revolutions per minute, about 2 more milliseconds pass before the data can be read. After the read-write head is positioned, the actual reading (or writing) process begins; this might take a few more milliseconds.

Thus, disk access times of around 10 milliseconds are common. This is something like 1,000,000 times slower than a main memory that could access a cell in 10 nanoseconds. The ratio varies considerably with the devices, but it's always large, say above 100,000.

Next, let's consider flash memory, another form of external storage. Like disks, it retains its data even when power is turned off. Flash is faster to access than spinning disks but still slower than main memory. A flash memory device might take something like 10 to 100 microseconds (millionths of a second) to access a particular part of the data. Although that's much faster than the time to get the disk head positioned, it still is 1,000 or 10,000 times more than reading a cell from main memory.

In cloud computing environments, blocks of data are distributed over networked servers. Accessing a particular block stored on server A to be processed on server B means copying the data over the network. Network speeds grow faster every few years but can vary considerably with the traffic load. The network access speed compared to the speed of access to main memory is likely to be millions or billions of times slower.

The technology changes quickly, and speeds and costs of both main memory and external storage will change. What is likely not to change is that the fastest memory remains costly, either in terms of the money needed to buy it or the energy it consumes, compared with the slower external memory systems. The difference in speed is likely to remain on the order of thousands or millions to one. Hence, we will always need effective ways of accessing the higher-capacity, lower-cost external storage.

One Block at a Time

For disk storage, when the read-write head is correctly positioned and the reading (or writing) process begins, the drive can transfer a large amount of data to (or from) main memory quickly. For this reason, and to simplify the drive control mechanism, data is stored on the disk in chunks called *blocks*, *pages*, *allocation units*, *segments*, or some other name, depending on the system. Flash memory also provides access in terms of pages or blocks even though it doesn't involve a spinning disk. We call the chunks of data *blocks* in this discussion. Notably, the chunks of storage do *not* correspond directly to files in external memory; files typically take many blocks.

The disk drive always reads or writes a minimum of one block of data at a time. Block size varies depending on the device and is usually a power of 2. For our contact database example, let's assume a block size of 8,192 bytes (2^{13}). That means a database of a million (1 kilobyte) records will require 1,024,000,000 bytes divided by 8,192 bytes per block, which is 125,000 blocks.

External memory accesses are most efficient when they read or write a multiple of the block size. If you ask to read 100 bytes, the system will read one block, 8,192 bytes, and ignore all but 100 of them. If you ask to read 8,200 bytes, it will read two blocks, or 16,384 bytes, and throw away almost half of them. The device must still spend time fetching the full block(s), regardless of the request size. By organizing your software so that it works with a full block or blocks of data at a time, you can optimize its performance.

Assuming the contact database record size is 1,024 bytes, you can store eight records in a block (8,192 divided by 1,024), as shown in [Figure 9-18](#). Thus, for maximum efficiency, it's important to read eight records at a time (or multiples of this number).

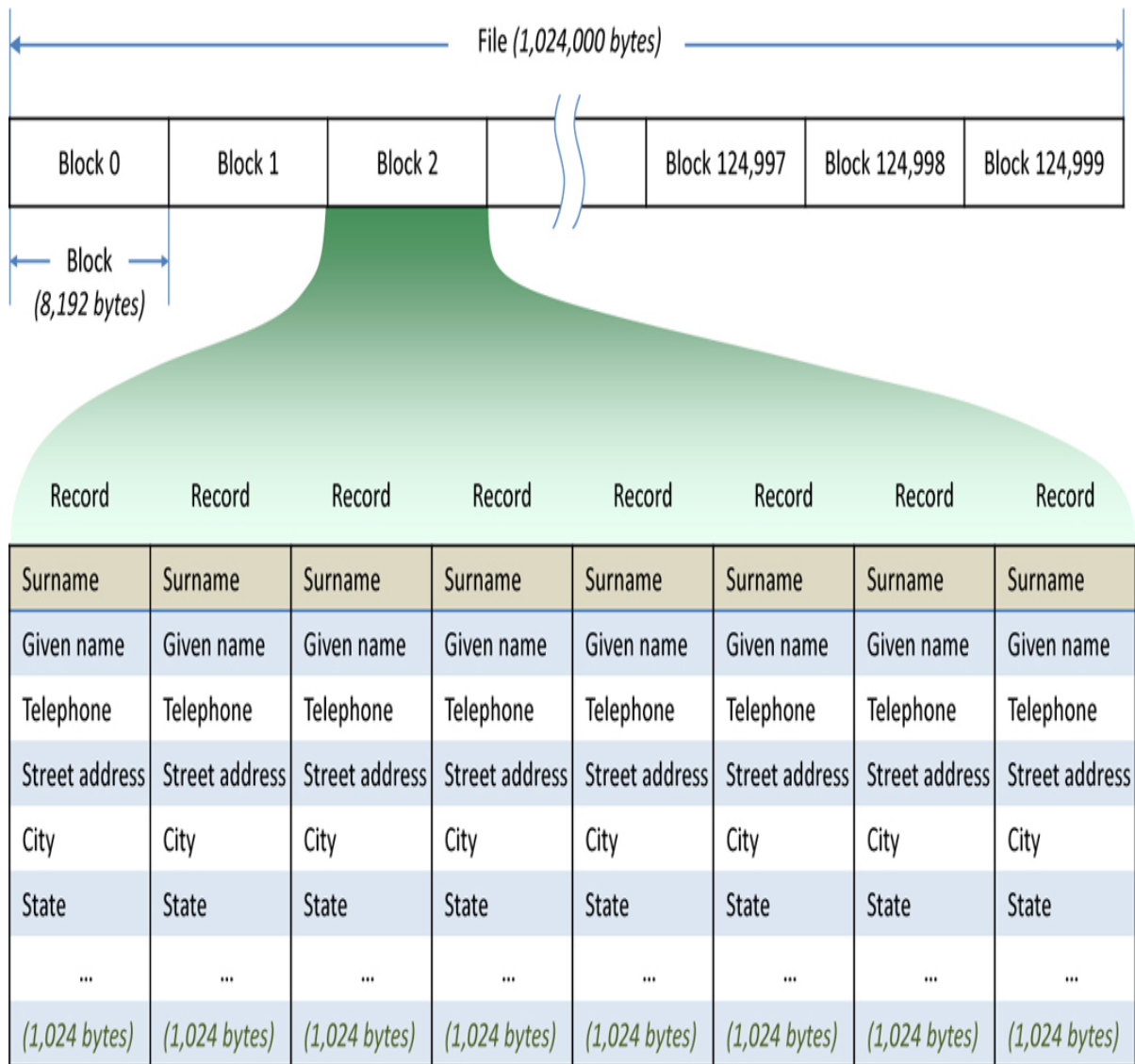


Figure 9-18 *Blocks and records in a file*

Notice that it's also useful to make your record size a power of 2 because that evenly divides the block size, which is typically a power of 2. In any case, it's best to have an integral number of records fit in a single block without any wasted space.

Of course, the sizes shown in the example for records, blocks, and so on are only illustrative; they will vary widely depending on the type, number, and size of records and other software and hardware constraints. Blocks containing hundreds of records are common, and records may be much larger or smaller than 1,024 bytes.

After the read-write head of a disk is positioned as described earlier, reading a block is fast, requiring only milliseconds. Thus, a disk access to read or write a block is not very dependent on the size of the block. It follows that the larger the block, the more efficiently you can read or write a single record (assuming you use all the records in the block).

Sequential Ordering

One way to arrange the database in a file on the disk would be to order all the records according to some key, say alphabetically by surname. Maybe the record for a person named Aaron Aardvark would come first, and so on. This scenario is illustrated in [Figure 9-19](#).

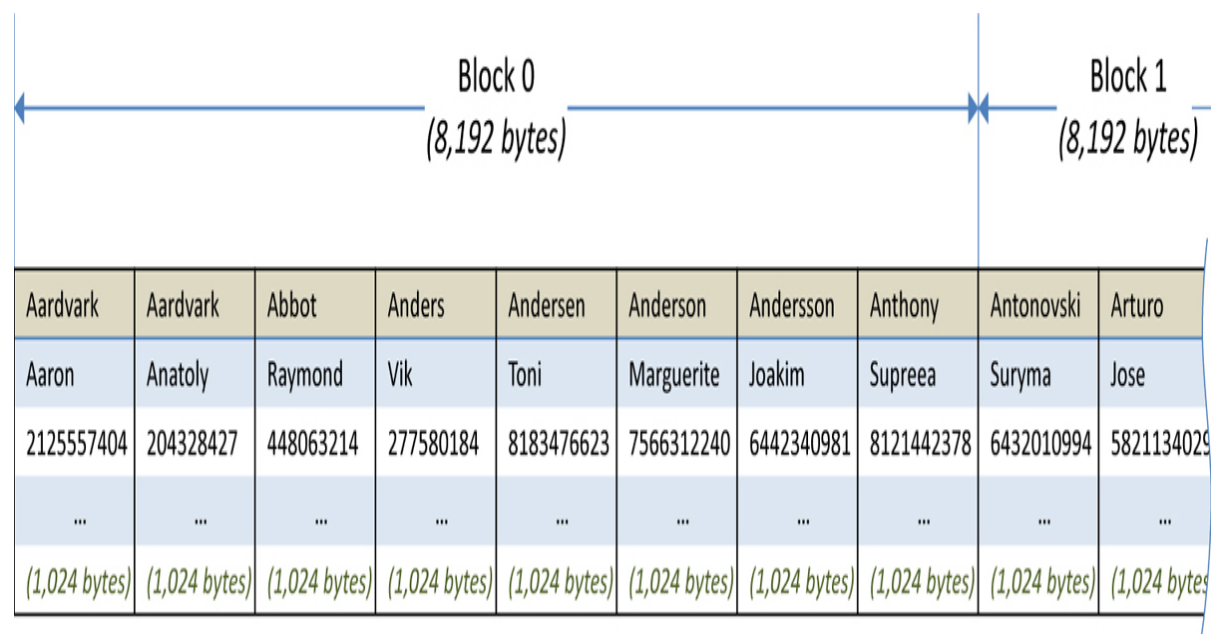


Figure 9-19 *Sequential ordering of records*

Searching

To search a sequentially ordered file for a particular surname such as Turing, you could use a binary search. You would start by reading a block of records from the middle of the file. Let’s simplify the discussion and assume all the blocks of the file are in sequentially ordered blocks on the disk, so it’s easy to find the middle block by knowing the first and last block numbers (this is rarely the case in actual file systems, but there are often ways to get the block

numbers almost as quickly). The eight records in the block are all read at once into an 8,192-byte array (or **buffer**) in main memory.

If the keys of these middle records were too early in the alphabet (ending with Kahn, for example), you would go to the 3/4 point in the file and read a block there, perhaps finding Pearl; if the keys in the middle records came after search key, you'd go to the 1/4 point, perhaps finding Englebart. By continually dividing the range in half, you would eventually find the target record.

As you saw in [Chapter 3](#), a binary search in main memory takes $\log_2 N$ comparisons, which for 1,000,000 items would be about 20. If every comparison took, say 1 microsecond, this would be 20 microseconds, or about 1/50,000 of a second, much less than an eye blink.

In this section, however, we're dealing with data stored on a disk. Because each disk access is so time-consuming, it's more important to focus on how many disk accesses are necessary than on how many individual records there are. The time to read a block of records will be very much larger than the time to search the eight records in the block once they're in memory.

Disk accesses are much slower than memory accesses, but on the other hand, you access the disk a block at a time, and there are far fewer blocks than records. In our example, there are 125,000 blocks. $\log_2$ of this number is about 17, so in theory, you need about 17 disk accesses to find the record you want.

In practice this number is reduced somewhat because you read eight records at once. In the beginning stages of a binary search, it doesn't help to have multiple records in memory because the next access will be in a distant part of the file. When you get close to the desired record, however, the next record you want may already be in memory because it's part of the same block of 8 records. Having the next record in memory may reduce the number of disk accesses by two or so. Thus, you need about 15 disk accesses ($17 - 2$), which at 10 milliseconds per access requires about 150 milliseconds, or 0.15 seconds to complete the binary search of sequentially stored records. This is much slower than in-memory access of about 1/50,000 of a second, but still not too bad.

Insertion

Unfortunately, the picture is much worse if you want to insert (or delete) an item (a record) from a sequentially ordered file. Because the data is ordered,

both operations require moving half the records on average and, therefore, about half the blocks.

Updating each block requires two disk accesses: one read and one write. When the insertion point is found, the block containing it is read into a memory buffer. The last record in the block is saved, and the appropriate number of records are shifted up to make room for the new one, which is inserted. Then the buffer contents are written back to the disk file as a block.

Next, the block following the insertion block is read into the buffer. Its last record is saved in main memory, all the other records are shifted up, and the last record from the previous block is inserted at the beginning of the buffer. Then the buffer contents are again written back to disk. This process continues until all the blocks beyond the insertion point have been rewritten.

Assuming there are 125,000 blocks, you must read and write (on average) 62,500 of them, which at 10 milliseconds per read and write requires more than 20 minutes to insert a single entry ($20 \text{ ms} \times 62,500 = 1,250 \text{ seconds} = 20.83 \text{ minutes}$). This isn't satisfactory by today's standards, especially if you have more than a few records to add.

Another problem with the sequential ordering is that it works quickly for only one key. Our example contact database is arranged by surnames. Suppose you wanted to search for a particular phone number or city. You can't use a binary search because the data is ordered by name. You would need to go through the entire file, block by block, using sequential access. This search would require reading an average of half the blocks, which would require about 10 minutes, which is very poor performance for a simple search. We obviously need a more efficient way to store large amounts of data in external memory.

B-Trees

How can the records kept in external memory be arranged to provide fast search, insertion, and deletion times? You've seen that trees are a good approach to organizing in-memory data. Do trees work with external memory such as files on a disk?

They do, but a different kind of tree must be used for external data than for in-memory data. The appropriate tree is a multiway tree somewhat like a 2-3-4 tree, but with many more data items per node; it's called a **B-tree**. B-trees were first conceived as appropriate structures for external storage by R. Bayer and E.

M. McCreight in 1972. (Strictly speaking, 2-3 trees and 2-3-4 trees are B-trees of order 3 and 4, respectively, but the term *B-tree* is often taken to mean many more children per node.) Notably, B-trees are *not* binary trees, despite the similarity in their names.

One Block Per Node

Why would one need so many items per node? You've seen that disk access is most efficient when data is read or written one block at a time. In a tree, the structure containing data is the node (whether it be a node in a B-tree, a 2-3-4 tree, a binary search tree, or other). It makes sense then to store an entire block of data in each node of the tree. This way, reading a node accesses a maximum amount of data in the shortest time.

How much data can be put in a node? The answer depends on the size of the items, of course. In the contact database example using sequential records, you could fit 8 1-kilobyte data records into an 8,192-byte block.

In a tree, however, you also need to store the links to other nodes (which means links to other blocks because a node corresponds to a block). In an in-memory tree, such as those discussed in previous chapters, these links are references to nodes in other parts of memory. For a tree stored in a disk file, the links are to block numbers in the file (from 0 to 124,999, in the million record database example). For block numbers, you can use an integer field. If the integer takes 4 bytes (32 bits), it can reference a little more than 2 billion possible blocks, which might be enough. Bigger databases and external memory devices with even more addressable blocks would need even larger block numbers.

Organizing the data as tree nodes means that you can no longer squeeze eight 1,024-byte records into a block because you need room for the links to child nodes. You could reduce the number of records to 7 to make room for the links that take 8×4 -bytes, for example, but it's most efficient to have an even number of records per node and fill up the block. Perhaps the record size doesn't need to be the full 1,024 bytes; maybe a field or two can be reduced in size. Let's assume that the database designers would accept records of 1,010 bytes by shortening some of the fields. That means a node can still contain 8 records along with 9 child links (one more than the number of data items) requiring 36 bytes (9×4). The node would have $8 \times 1,010$ -byte records plus 36 bytes of links and a couple more 4-byte integers that hold the number of keys and child links (`nKeys` and `nChild`), leaving 68 bytes unused ($8,192 - 8,080 -$

36 – 8). A block in such a tree (and the corresponding full node representation) is shown in Figure 9-20.

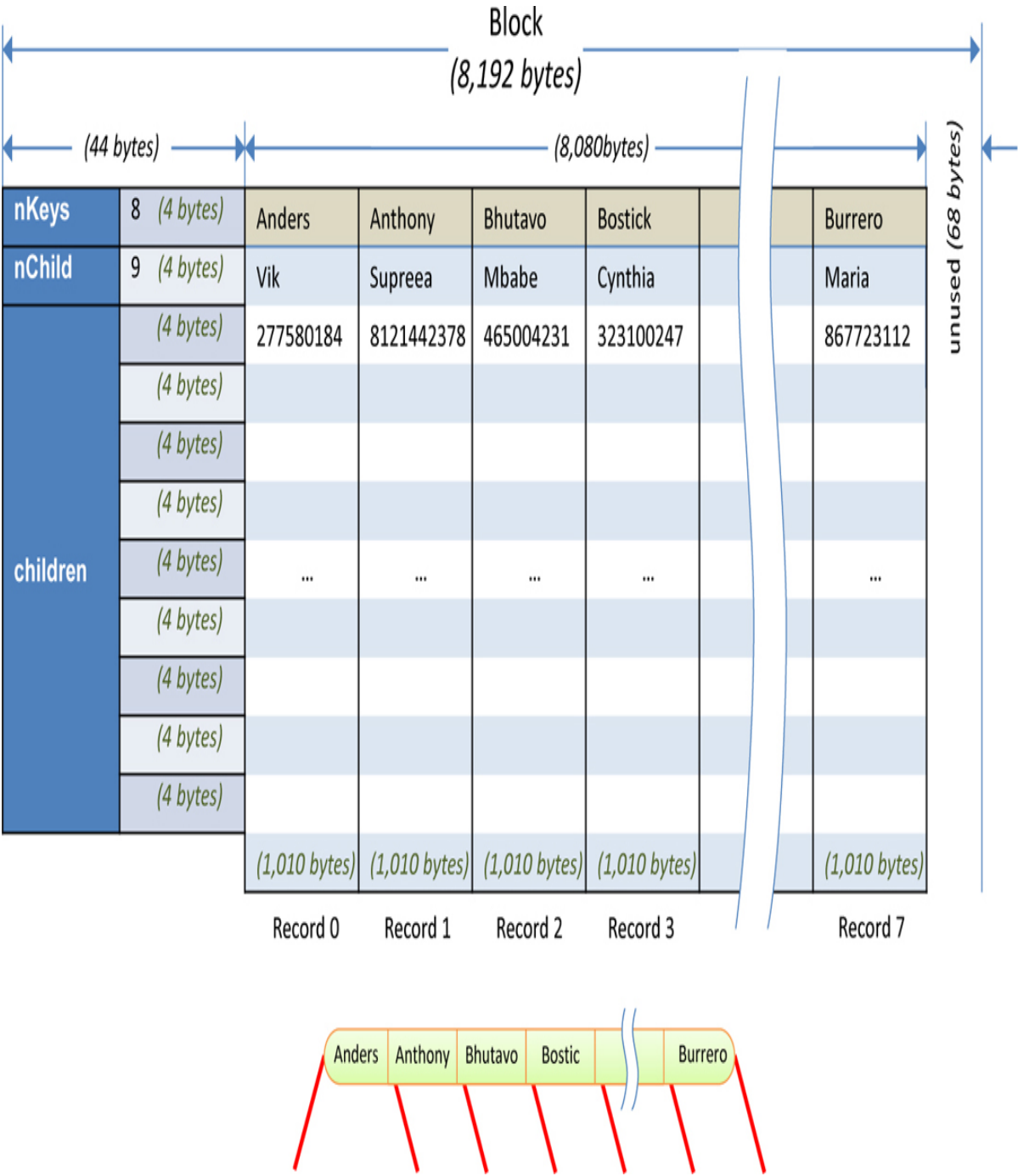


Figure 9-20 A node in a B-tree of order 9 with its block structure

Within each node the data is ordered sequentially by key, as in a 2-3-4 tree. In fact, the structure of a B-tree is like that of a 2-3-4 tree, except that there are

more data items per node and more links to children. The order of a B-tree is the number of children each node can potentially have. In the example this is 9, so the tree is an order 9 B-tree.

If the full tree structure will be stored in external memory, which is what is typically done for a database, you also need to store the object that represents the tree. The `Tree234` object is used to hold that information in the in-memory implementation. All that must be stored in the corresponding B-tree object is the reference to the block of the root node. There probably would be other attributes stored for a large database that would go in this object. That information would probably be stored in a block all by itself, or perhaps in place of the first record of a block. Even though there would likely be a lot of unused bytes in the block, it's much more efficient in terms of disk accesses to keep it in a separate block and treat all tree nodes identically.

Searching

A search for a B-tree record with a specified key is carried out in much the same way it is in an in-memory 2-3-4 tree. Let's assume the tree object is already in memory with its reference to the block containing the root node. First, the block containing that root is read into memory. The search algorithm then starts examining each of the eight records (or, if it's not full, as many as the node holds), starting at 0. When it finds a record with a greater key, it knows to go to the child whose link lies between this record and the preceding one. Note that you could use a binary search within the keys of a node, and it makes more sense to do so for large numbers of keys. Using binary search saves some in-memory comparisons, which is only a small savings compared to the time needed to read the external node/block.

This process continues until the correct node is found. If a leaf is reached without finding the specified key, the search is unsuccessful.

Insertion

The insertion process in a B-tree is more like an insertion in a 2-3 tree than in a 2-3-4 tree. Recall that in a 2-3-4 tree many nodes are not full and, in fact, contain only one data item. A 2-3-4 node split always produces two nodes with one item in each. This is not an optimum approach in a B-tree, even though splits don't have to create nodes with a single item.

In a B-tree it's important to keep the nodes as full as possible so that each disk access, which reads an entire node, can acquire the maximum amount of data. To help achieve this end, the insertion process differs from that of 2-3-4 trees in three ways:

- A node split divides the data items equally: half go to the newly created node, and half remain in the old one.
- Node splits are performed from the bottom up, as in a 2-3 tree, rather than from the top down.
- Again, as in a 2-3 tree, it's not the middle item in a node that's promoted upward, but the middle item in the sequence formed from the items in the node plus the new item.

We demonstrate these features of the insertion process by building a small B-tree, as shown in [Figure 9-21](#). There isn't room to show a realistic number of records per node, so we show only eight; thus, the tree is an order 9 B-tree. We also switch to using integer keys to fit them all in the figure.

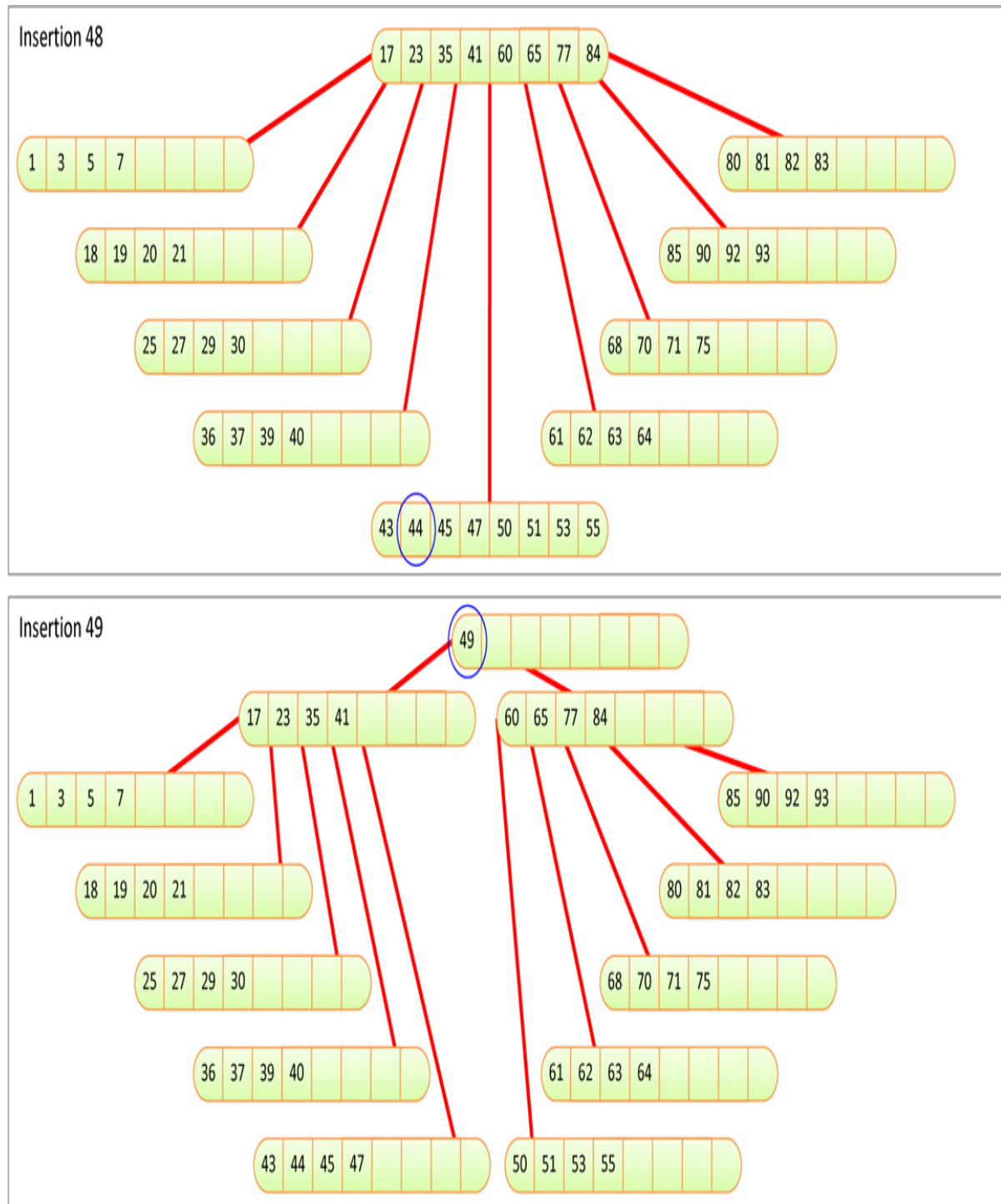


Figure 9-21 *Building a B-tree*

At the top left of [Figure 9-21](#) is a panel labeled Insertion 1. It shows a root node after the first item, with key 60, has been inserted. The blue circle indicates the item that was just inserted. Below that is Insertion 8, where the

eighth item, with key 40, has been inserted. This fills all the item slots in the root node. As usual, all the items are stored in sorted order.

In the next panel, Insertion 9, a new data item with a key of 75 is inserted, resulting in a node split. Here's how the split is accomplished. Because the root is being split, two new nodes are created (as in a 2-3-4 tree): a new root and a new node to the right of the one being split.

To decide where the data items go, the insertion algorithm arranges their nine keys in order, in an internal buffer. Eight of these keys are from the node being split, and the ninth is from the new item being inserted. This would go between item 70 and 80, leaving 60 as the middle item with four others on either side.

The middle item 60 is placed in the new root node. All the items to the left of it remain in the node being split, and all the items to the right go into the new right-hand node. In Insertion 9, the inserted item 75 ends up in the split leaf node, but that's not always the case, as you can see in later examples.

The number of items in a split node is always half that of a full node. So, if this were an order 31 B-tree, 15 items would go in the new split node. The single middle item would be promoted to the parent, and the original leaf node would be reduced to 15 items too.

It's clear after Insertion 9 that the B-tree has capacity to store many more items in the existing three nodes. Depending on their keys, newly inserted items go in one of the two leaf nodes. Let's insert six more items with keys 35, 65, 83, 55, 92, and 77. Two of those items, 35 and 55, go in the left leaf and the other four go in right leaf. That brings the right leaf to full capacity with 8 items, and the overall B-tree has 15 items in it.

On Insertion 16, item 71 arrives, and it must go in the right leaf. Because that leaf is full, it must be split. Item 71 lands before the middle of the eight items in that right leaf (65, 70, 75, 77, 80, 83, 90, 92). That means item 71 won't go into the new split node; it will stay with the "current" node being split. The four items with higher keys are split from the current node to form the new node. Item 77 is taken out of the current node and promoted to its parent, the root. The current node becomes child 1 of the root, and the new split node is child 2. In this case, the inserted item ends up in the left part of the split. The resulting B-tree is shown in the upper right panel of [Figure 9-21](#). It has four nodes, and the 16th inserted item 71 ended up in the central leaf.

Promoting item 77 from the leaf to the root doesn't cause more splits because the root only had a single item in it. Note that the three leaf nodes after Insertion 16 are all at the same level of the tree, level 1, even though we show them at different vertical positions in the figure.

The next insertions continue to add items to the leaves according to their keys. It takes at least five more inserts to split the newest leaves because they start off with four items each. As the splits occur, one item gets promoted to the parent, the root node. After at least 28 more inserts, the root can have 9 child nodes each holding 4 items (44 items total = $9 \times 4 + 8$). After adding at least 4 more items, one of child nodes can grow to holding 8 items and become full. That brings the total to 48 items spread over the 10 nodes.

Insertion 49 of [Figure 9-21](#) shows how a split could happen that expands the B-tree to two levels. For this to happen, the inserted item must go in a full leaf under a full root. Let's assume, somewhat coincidentally, that an item with key 49 is inserted. This item lands in a full leaf containing items 43, 44, 45, 47, 50, 51, 53, and 55. That node must split and, in this case, because 49 lands in the middle of the nine items, item 49 is promoted instead of one of the previously inserted items. That splits the full leaf node into two 4-item leaf nodes shown at the bottom of the panel labeled Insertion 49 in [Figure 9-21](#).

The promoted item 49 goes up to the root node, but because that is also full, the node must be split. Looking at the promoted item in relation to the items in the root, 17, 23, 35, 41, 60, 65, 77, and 84, shows that it again lands in the middle. Thus, item 49 gets promoted again, and the full root node is split in two. That is how you end up at Insertion 49 of [Figure 9-21](#). Item 49 is promoted out of the root node so that it forms a new single item node. The old root is split into nodes 17-23-35-41 and 60-65-77-84. This (contrived) example shows how the inserted node could end up at an internal node or even in a new root. All the leaves remain at the same level in the tree, despite their positions in the figure.

Notice that throughout the insertion process no node (except the root) is ever less than half full, and many are more than half full. As we noted, keeping nodes as full as possible promotes efficiency because a file access that reads a node always acquires a substantial amount of data. The nodes are not split on the downward search to the node where the item should be inserted; they are only split as the insertion encounters a full leaf node and promotes the overflow back up toward the root. Thus, no unneeded splits cause expensive disk accesses.

Efficiency of B-Trees

Because there are so many records per node, and so many nodes per level, operations on B-trees are very fast, considering that the data is stored on slower storage like a disk. In the contact database example, there are a million records. All the nodes in the B-tree other than the root are at least half full, so they contain at least 4 records and 5 links to children. The height of the tree is thus somewhat less than $\log_5 N$ (logarithm to the base 5 of N), where N is 1,000,000. This is about 8.58, so there will be 9 levels in the tree (height 8).

Thus, using a B-tree, only nine disk accesses are necessary to find any record in a file of 1,000,000 records. At 10 milliseconds per access, this takes about 90 milliseconds, or 9/100 of a second. This is dramatically faster than the binary search of a sequentially ordered file.

The more records there are in a node, the fewer levels there are in the tree. You've seen that there are 9 levels in our example B-tree, with the nodes holding only 8 records each. In contrast, a binary tree with 1,000,000 items would have about 20 levels, and a 2-3-4 tree would have 10. If you use blocks with hundreds of records, you can reduce the number of levels in the tree and further improve access times, although this is an option only for devices that support blocks large enough to hold all those records.

Although searching is faster in B-trees than in sequentially ordered disk files, it's insertion and deletion that demonstrate B-trees greatest advantages.

Let's first consider a B-tree insertion in which no nodes need to be split. This is the most likely scenario because of the large number of records per node. In our contact database example, at most 9 block accesses are required to find the insertion point. Then one more access can write the block containing the newly inserted record back to the disk, for a total of 10 accesses.

Next let's see how things look if a node must be split. The node being split must be read, have half its records removed, and be written back to disk. That's the same as what was needed for an insertion with no split, 10 accesses. The newly created node must be written to the disk, and the parent must be read and, following the insertion of the promoted record, written back to disk. This adds 3 accesses to the 10 of the simpler case, for a total of 13 (although the parent node might remain in main memory after being read on the search for the insertion point, so the total could be 12). This number is a major improvement over the 500,000 accesses required for insertion in a sequential

file. As splits are promoted up the B-tree, they each add 3 more accesses per level, so at most $7 \times 3 = 21$ more accesses in an 8-level tree.

In some versions of the B-tree, only leaf nodes contain records. Nonleaf nodes contain only keys and block numbers. This strategy may result in faster operation because each block can hold many more block numbers. The resulting higher-order tree will have fewer levels, and access speed will be increased. On the other hand, programming may be complicated because there are two kinds of nodes, leaves with values and nonleaves without them, and the need to find a leaf node corresponding to an internal node's key to access the value.

Indexing

A different approach to speeding up external memory access is to store records in sequential order but use a **file index** along with the data itself. A file index is a list of key/block pairs, arranged with the keys in order. Recall that our original contact database example had 1,000,000 records of 1,024 bytes each, stored 8 records to a block, in 125,000 blocks. To make a file index using surnames as the key, every entry in the index contains two items:

- The key, a string like Jones.
- The number of the block where the Jones record is located within the file. These numbers run from 0 to 124,999.

Let's say you use a string 59 bytes long for the key (big enough for most surnames including Unicode characters), 4 bytes for the block number (an integer), and a byte for the record number within the block. Each entry in such an index would require 64 bytes. This is only 1/16 the amount necessary for each record.

The entries in the index are arranged sequentially by surname. There could be multiple entries for common surnames. The original records on the disk can be arranged in any convenient order. This usually means that new records are simply appended to the end of the sequence, so the records are ordered by time of insertion. This arrangement is shown in [Figure 9-22](#).

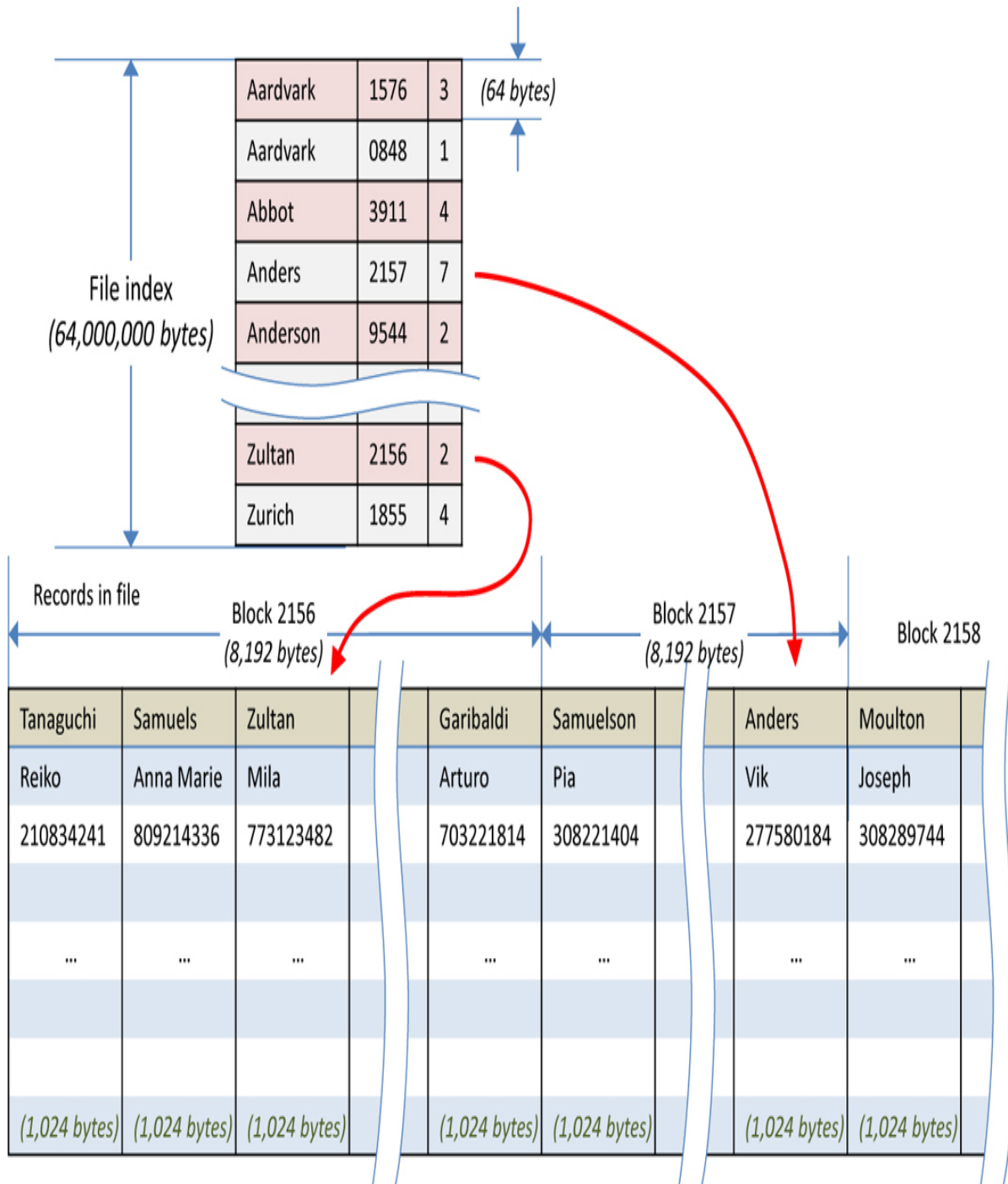


Figure 9-22 *A file index example*

File Index in Memory

The file index is much smaller than the file containing actual records. It may even be small enough to fit entirely in main memory. In this example there are

1,000,000 records. Each one has a 64-byte entry in the index, so the index will be $64 \times 1,000,000$, or 64,000,000, bytes long (a little less than 64 megabytes). In modern computers there's no problem fitting this index in memory (although it could be too big for a very tiny computer in some miniature device).

The file index can be stored in external memory and read into main memory whenever the database program is started up. From then on, operations on the index can take place in memory. At the end of each day (or whenever the database is shut down), the index can be written back to disk for permanent storage. The 64,000,000-byte index would take up 7,813 blocks on the disk. That might require several seconds to read or write, but only at startup and shutdown.

Searching

The *index-in-memory* approach allows much faster operations on databases than are possible with records arranged sequentially in external memory. For example, a binary search could require 20 index accesses. Even at 0.1 microsecond (100 nanoseconds) per access, that's only about $2/1,000,000$ of a second. There's inevitably the time needed to read the actual record from external memory, after its block number has been found in the index. On a disk, that read operation might take one block access or about 10 milliseconds (1/100 of a second).

Insertion

To insert a new item in an indexed file, two steps are necessary. You first insert the item's full record into the main file; then you insert an entry, consisting of the key and the block number where the new record is stored, into the file index.

Because the file index is in sequential order, to insert a new item, you need to move half the index entries, on average. Figuring about 3 nanoseconds to move a byte in memory (one read and one write of 8 bytes takes about 20 nanoseconds), you have 500,000 entries times 64 bytes per entry times 3 nanoseconds per byte, or about .096 ($1/10^{\text{th}}$) seconds to insert a new entry. This compares with 20 minutes for updating the unindexed sequential file. Note that you don't need to move any records in the main file; you simply append the new record at the end of the file. You would eventually need to write the file

index to external memory, which would take about 72 seconds, writing another 7,182 ($= 1,000,000 \times 64 / 8,192$) blocks at $1/100^{\text{th}}$ of a second per block.

Of course, you can use a more sophisticated approach to storing the file index in memory. You could store it as a binary tree, 2-3-4 tree, or another multiway tree, for example. Any of these would significantly reduce insertion and deletion times. In any case the index-in-memory approach is much faster than the sequential-file approach. In some cases, it is also faster than a B-tree.

The only actual disk accesses necessary for an insertion into an indexed file involve the new record itself and eventually storing the file index. Usually, the last block in the file is read into memory, the new record is appended, and the block is written back out. This process involves only two file accesses. Storing the file index means writing thousands of blocks, but that is needed only when the database service is being shut down. The time spent preserving the file index is spread out over the hundreds, thousands, or perhaps millions of insertions and searches.

Multiple Indexes

An advantage of the indexed approach is that multiple indexes, each with a different key, can be created for the same set of records (database). In one index the keys can be last names; in another, telephone numbers; in another, addresses. Because the indexes are small compared with the data file, this doesn't increase the total data storage very much. Of course, it does present more of a challenge when items are inserted or deleted from the file because entries must be added to or deleted from all the indexes, but we don't get into that here.

Index Too Large for Memory

If the index is too large to fit in memory, it too must be broken into blocks and stored on the disk. For large files, storing the index itself as a B-tree may then be profitable. In the main file, the records are stored in any convenient order.

This arrangement can be very efficient. Appending records to the end of the main file is a fast operation, and inserting the index entry for the new record is also quick because the index is a tree. The result is very fast searching and insertion for large files.

Note that when a file index is arranged as a B-tree, each node contains n child pointers and $n-1$ data items. The child pointers are the block numbers of other nodes in the file index. The data items consist of a key value and a pointer to a block and record in the main file. Don't confuse these two kinds of block pointers.

Complex Search Criteria

In complex database searches, the only practical approach may be to read every block in a file sequentially. Suppose in our contact database example you wanted a list of all entries with first name Kristen, who lived in Sao Paolo, and who had a phone number with the digits 284 in it. Imagine the name and phone number were found on a scrap of paper in a bag with a lot of money somewhere in Sao Paolo, and it's important to locate the full record for the person.

A file organized by last names would be no help at all. Even if there were index files ordered by first names and cities, there would be no convenient way to find which files contained both Kristen and Sao Paolo. Even less likely are indexes for three-digit subsequences of phone numbers (although they are sometimes created). In such cases the fastest approach is probably to use one of the indexes, if it exists, and read every record it references for the particular name, block by block, checking each record to see whether it meets the rest of the criteria.

Sorting External Files

You've seen how to use B-trees to manage a database stored in external memory. The data for each node, and hence each block, is sorted by a key. The sorting happens as items are inserted. What about when some other process wrote data to files, and you want to sort that data? Imagine a bunch of log files written by some process on a computer. Each record in the log probably has a date and time, along with other information that was relevant to the process such as the phase of the operations, the type of event that occurred, the size of some internal data structure, the account numbers for transactions, and so on. If you want to sort the log files by some field other than date and time (assuming they were recorded chronologically), and they don't all fit into internal memory, what's the best way to sort them?

Mergesort is the preferred algorithm for sorting large amounts of external data. The reason is that, more so than most sorting techniques, disk accesses tend to occur in adjacent records rather than random parts of the file.

Recall from [Chapter 6](#), “[Recursion](#),” that mergesort works recursively by calling itself to sort smaller and smaller sequences. After two of the smallest sequences (one element each in the internal-memory version) have been sorted, they are then merged into a sorted sequence twice as long. Larger and larger sequences are merged, until eventually the entire file is sorted.

The approach for external storage is similar. However, the smallest sequence that can be read from the disk is a block of records. Thus, a two-stage process is necessary.

In the first phase, a block is read, its records are sorted internally, and the resulting sorted block is written back to disk. The next block is similarly sorted and written back to disk. This process continues until all the blocks are internally sorted.

In the second phase, two sorted blocks are read, merged into a two-block sequence, and written back to disk. This process continues until all pairs of blocks have been merged. Next, each pair of two-block sequences is merged into a four-block sequence. In each step, the size of the sorted sequences doubles, until the entire file is sorted.

[Figure 9-23](#) shows the mergesort process on an external file. The file consists of eight blocks of 16 records each, for a total of 128 records. Instead of showing a record with a key in the figure, the records are shown as colored rectangles. The color and height of each rectangle are its key. Even though they are of different heights in the figure, the records on the disks have the same length in terms of bytes.

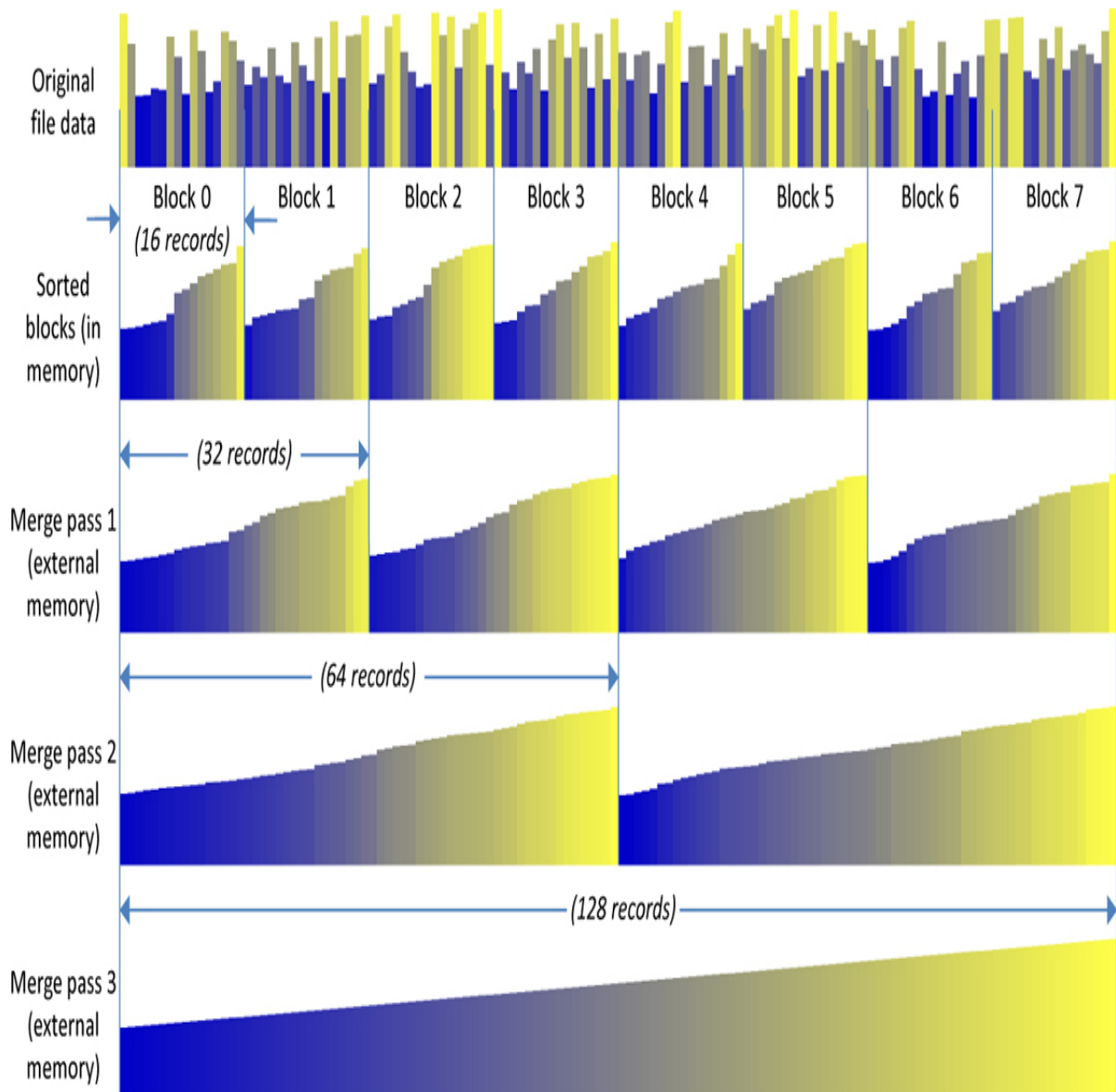


Figure 9-23 *Mergesort on an external file*

Let's assume that only three blocks can fit in internal memory, so you can't simply read all the records at once, sort them, and write them all out. Of course, all these sizes would be much larger in a real situation. The first row of [Figure 9-23](#) shows the original, unsorted file data.

Internal Sort of Blocks

In the first phase all the blocks in the file are sorted internally. Each block is read into memory and then its records are sorted with any appropriate internal sorting algorithm, such as quicksort (or, for smaller numbers of records,

Shellsort or insertion sort). The internal sort could also be done with mergesort because the main disadvantage of mergesort is not an issue here. Remember that mergesort needs a second array the size of the original but that buffer must be available because mergesort needs it to merge two blocks later in memory. Whatever sorting algorithm is used, the sorted blocks of 16 records each are written back to external memory as shown in row 2 of [Figure 9-23](#).

A second file, File 2, may be used to hold the sorted blocks, under the assumption that the availability of external storage is not a problem. It's often desirable to avoid modifying the original file.

Merging

In the second phase, the sorted blocks are merged. The first pass merges every pair of 16-record blocks into a sorted two-block sequence. Thus, the leftmost two blocks, 0 and 1, are merged into the leftmost 32 records in merge pass 1. Also, blocks 2 and 3, blocks 4 and 5, and blocks 6 and 7 are merged into their respective sequences of 32 records. The result is shown in the merge pass 1 row of [Figure 9-23](#). Let's assume that the process writes a third file, file 3, to hold the result of this merge step (although that may not always be necessary).

In merge pass 2, two 32-record sequences are merged into a 64-record sequence, which can be written back to file 2. There are four of these sequences that get merged into two larger sequences. The last row of [Figure 9-23](#) shows the result of merge pass 3 where the two 64-record sequences are merged into a 128-record file, file 3. Now the sort is complete. Of course, more merge steps would be required to sort larger files; the number of such steps is proportional to $\log_2 N$. The merge steps can alternate between two files, with one being discarded at the end.

Internal Arrays

Because the computer's internal memory has room for only three blocks, the merging process must take place in stages. Let's say there are three arrays, called `arr0`, `arr1`, and `arr2`, each of which can hold a block.

In merge pass 1, block 0 is read into `arr0`, and block 1 is read into `arr1`. These two arrays are then mergesorted into `arr2`. Because `arr2` holds only one block, it becomes full before the sort is completed. When it becomes full, its contents are written to disk in file 3. The sort then continues, filling up `arr2` again. This completes the sort, and `arr2` is again written to disk. The process moves on to

blocks 2 and 3, reading them into `arr0` and `arr1` and merging their contents into `arr2`. As `arr2` becomes full, the results are written to the end of file 3.

In merge pass 2, you can continue using the three arrays, `arr0`, `arr1`, and `arr2`, even though the lengths of the sorted sequences have exceeded the array size. The input arrays, `arr0` and `arr1`, are initially filled with the first block of the sequence. After all the records from, say, `arr0` have been merged into the output array, `arr2`, the next block from that sequence can be read to refill `arr0`. You use arrays that are the size of a block (or perhaps a few blocks depending on how much memory is available) to efficiently read and write the external memory. Their size doesn't depend on the full amount of data to sort. This scheme works for all the merge passes.

We can make another important simplification to the external mergesort algorithm. In the first phase, it wrote the sorted blocks back to external memory. Because the first step of merge pass 1 is reading those blocks back into `arr0` and `arr1`, we can eliminate the first phase by performing the in-memory sort right after reading the unsorted block. The sort only happens on merge pass 1, and all the other merge passes only perform merges.

Efficiency of External Mergesort

The overall efficiency of mergesort remains $O(N \times \log N)$, even when the data is in external memory. Due to the slow access to external data, however, the running time is much longer than if all the data could fit in internal memory. The number of external data accesses becomes the controlling factor. In the second phase, every block of the File 2 is read, sorted if it's merge pass 1, merged appropriately, and then written back to the external store. Thus, there are two accesses (a read and write) for every block in every merge pass. That brings the total to two times the number of merge passes. The number of merge passes is proportional to $\log_2 N$ but is reduced by the fact that blocks contain multiple records, say B records per block. That makes the number of merge passes $\log_2 N/B$, or due to the way logarithms work, $\log_2 N - \log_2 B$. With the external memory accesses taking thousands or millions of times longer than the in-memory operations, they dominate the overall time.

Summary

- A multiway tree has more items and children per node than a binary tree.

- A 2-3-4 tree is a multiway tree with up to three items and four children per node.
- In a multiway tree, the items in a node are arranged in ascending order by their keys.
- In a 2-3-4 tree, all insertions are made in leaf nodes, and all leaf nodes are on the same level.
- Three kinds of internal nodes are possible in a 2-3-4 tree: A 2-node has one item and two children, a 3-node has two items and three children, and a 4-node has three items and four children.
- There is no 1-node in a 2-3-4 tree.
- In a search of a 2-3-4 tree, at each node the keys are examined. If the search key is not found, the next node will be child 0 if the search key is less than key 0; child 1 if the search key is between key 0 and key 1; child 2 if the search key is between key 1 and key 2; and child 3 if the search key is greater than key 2.
- Insertion into a 2-3-4 tree requires that any full node be split on the way down the tree, during the search for the insertion point.
- Splitting the root creates two new nodes; splitting any other node creates one new node.
- The height of a 2-3-4 and a 2-3 tree can increase only when the root is split.
- The heights of 2-3-4 and 2-3 trees are less than $\log_2(N)$.
- Search times are proportional to the tree height.
- The 2-3-4 tree wastes space because many nodes might not even be half full.
- A 2-3 tree is like a 2-3-4 tree, except that it can have only one or two data items and zero, two, or three children.
- Insertion in a 2-3 tree involves finding the appropriate leaf and then performing splits from the leaf upward, until a non-full node is found.

- Both 2-3 and 2-3-4 trees maintain balance during inserts and deletes by keeping all leaf nodes on the same level.
- External storage means storing data outside of main memory, such as on a disk.
- External storage is larger, cheaper (per byte), and slower than main memory.
- Data in external storage is typically transferred to and from main memory a block at a time.
- Blocks can be different sizes but are typically hundreds to tens of thousands of bytes.
- Accessing an arbitrary block of external memory is thousands or millions of times slower than accessing arbitrary internal memory.
- Although accessing an arbitrary block is slow, accessing consecutive blocks is typically much faster.
- Data can be arranged in external storage in sequential key order. This gives fast search times but slow insertion (and deletion) times.
- A B-tree is a multiway tree in which each node may have dozens or hundreds of keys and children. The number of possible children is the order of the B-tree.
- There is always one more child than there are items in a B-tree, 2-3 tree, or 2-3-4 tree internal node. Leaf nodes have no child links.
- For the best performance with external memory, a B-tree is typically organized so that a node holds one block of data.
- Indexes to data stored in external memory provide fast search times and allow for multiple keys to be used in searches.
- If the search criteria involve many keys, a sequential search of all the records in a file may be the most practical approach.
- External mergesort is an efficient way to sort data stored in external memory.

- By using three arrays sized as (a small multiple of) the block size in external memory, mergesort can sort more data than can fit in internal memory.

Questions

These questions are intended as a self-test for readers. Answers may be found in [Appendix C](#).

1. A 2-3-4 tree is so named because a node can have
 - a. three children and four data items.
 - b. zero, two, three, or four children.
 - c. two parents, zero or three children, and four items.
 - d. two parents, three items, and zero or four children.
2. A 2-3-4 tree is superior to a binary search tree in that it is _____.
3. Imagine a parent 2-3-4 node with item keys 25, 50, and 75. If one of its child nodes had items with values 60 and 70, it would be the child numbered _____.
4. True or False: Data items in a 2-3-4 tree are located exclusively in leaf nodes.
5. Which of the following is **not** true every time a node below the root in a 2-3-4 tree is split?
 - a. Exactly one new node is created.
 - b. Exactly one new data item is added to the tree.
 - c. One data item moves from the split node to its parent.
 - d. One data item moves from the split node to its new sibling.
6. A 2-3-4 tree increases its number of levels when _____.
7. Searching a 2-3-4 tree does **not** involve
 - a. splitting nodes on the way down if necessary.
 - b. picking the appropriate child to go to, based on the keys of items in a node.

- c. ending up at a leaf node if the search key is not found.
 - d. examining at least one key in any node visited.
8. After a nonroot node of a 2-3-4 tree is split, which item does its new right child contain, the item previously numbered 0, 1, or 2?
9. Which of the following statements about a node-splitting operation below the root of a 2-3 tree (not a 2-3-4 tree) is **not** true?
- a. The parent of a split node must also be split if it is full.
 - b. The item with the smallest key in the node being split always stays in that node.
 - c. The item being inserted at a leaf or the item promoted from a lower split must be compared with the other items of the node being split.
 - d. The splitting process starts at a leaf and works upward.
10. What is the Big O efficiency of inserting and deleting an item in a 2-3 tree?
11. In accessing data on a disk drive,
- a. merging sorted records is not always possible because at least half the records must be in RAM to do so efficiently.
 - b. moving data to make room to insert records is fast because so many items can be accessed at once.
 - c. deleting a record is fast because it can be marked as available for use by other programs.
 - d. reading two consecutive records could be 10,000 times faster than reading two random records in a large file.
12. In a B-tree for external storage, each node contains _____ data items.
13. Node splits in a B-tree are most like node splits in a _____ tree.
14. In external storage, indexing means keeping a file of
- a. keys and their corresponding blocks and records.
 - b. records and their corresponding blocks.
 - c. keys and their corresponding records.
 - d. last names and their corresponding keys.

15. When sorting data in external storage that is too large to fit in memory, the most efficient way is
- a. creating a 2-3-4 tree for each block and traversing it in order.
 - b. copying half the data in memory, sorting it, and then merging in the rest a block at a time.
 - c. using block size arrays and mergesorting.
 - d. quicksorting each block first and then using an insertion sort on blocks.

Experiments

Carrying out these experiments will help to provide insights into the topics covered in the chapter. No programming is involved.

- 9-A Draw by hand what a 2-3-4 tree looks like after inserting each of the following keys: elm, asp, oak, fig, bay, fir, gum, yew, and ash. Which item insertions cause splits? Don't use the Tree234 Visualization tool.
- 9-B Draw by hand what a 2-3 tree looks like after inserting the same sequence of values as in Experiment 9-A.
- 9-C Think about how you would remove a node from a 2-3 tree. What are all the cases that need to be handled?

Programming Projects

Writing programs to solve the Programming Projects helps to solidify your understanding of the material and demonstrates how the chapter's concepts are applied. (As noted in the Introduction, qualified instructors may obtain completed solutions to the Programming Projects on the publisher's website.)

- 9.1 Extend the definition of the `Tree234` class to include the following methods. Show how they operate on trees with 0, 1, and at least 10 items. Three of these methods don't need to explore all nodes, and at most only one of them should use the `traverse()` generator to be the most efficient.
- a. `levels()`—Count the number of levels in the tree. An empty tree and a tree with a single node and no children have zero levels.

- b. `nodes()`—Count the number of nodes in the tree. An empty tree has zero nodes. A single root node has one node.
- c. `items()`—Count the number of items in the tree. An empty tree has zero items. A single root node can have one, two, or three items.
- d. `minItem()`—Return the key and data of the item with the minimum key in the tree. Calling this on an empty tree should raise an exception.
- e. `maxItem()`—Return the key and data of the item with the maximum key in the tree. Calling this on an empty tree should raise an exception.

9.2 Two things are needed to build an index file as shown in [Figure 9-22](#): an assignment of records to blocks in external storage, and a sorted array of record keys. When you build a 2-3-4 tree, each of the records goes in a 2-3-4 node. You can assign to each 2-3-4 node a “block” number by writing a traversal method that returns each key and data along with a block number for the node it’s in and the index number of the data within the block. Block numbers can start at 1 and increment by 1.

For the second part, write a `SortedArray` class that maintains a set of records in sorted order. The `SortedArray` constructor should take a parameter that is a function to extract the sorting key from each record. The function can be called on each record to get the sorting key. The `SortedArray` class should have

- a. a `__len__()` method so that the number of records it holds can be found with the `len()` function.
- b. a `get(i)` method that returns the i^{th} record in the array
- c. a `find_index()` method that returns the index of a record containing a particular key, or if the key is not in the sorted array, the index where the new key would be inserted. This method should use binary search to find the index and stop when the first of multiple duplicate keys is found.
- d. a `search()` method that returns a record associated with a goal key or `None` if no record has such a key.
- e. an `insert()` method to add a new record to the sorted array (and should allow duplicate keys in records).

- f. a `delete()` method to delete a record from the sorted array that takes a full record as a parameter so that the exact record can be deleted from the array even when there are other records with the same key present.

Use the `SortedArray` and the block traversal method to create an index of a 2-3-4 tree built using the `Tree234` class described earlier. Insert the following key-value pairs into the 2-3-4 tree and then build the sorted index using the value (a year) as the sorting key. The 2-3-4 tree uses the first element of the following tuples as the record key and the second element as the data. Your `SortedArray` should then sort them by the second element (a year).

```
("Fran", 2006), ("Amir", 1996), ("Herb", 1975), ("Ken", 1979),  
("Ivan", 1988), ("Raj", 1994), ("Don", 1974), ("Ron", 2002),  
("Adi", 2002), ("Len", 2002), ("Vint", 2004), ("Tim", 2016)
```

Show the contents of the 2-3-4 tree and all the records in the `SortedArray` in sorted order. Print a file index for the years in the records. Each entry in the file index has a key (year), block number (2-3-4 node number), and record index. After printing the full index, delete a couple of items with duplicate year keys and show what records remain in the `SortedArray`.

9.3 A 2-3-4 tree can be used as a sorting machine. Write a `sortarray()` function that's passed an array of values and writes them back to the array in sorted order.

This project is somewhat complicated by the issue of duplicate values because the 2-3-4 tree expects unique keys. You can handle them, however, by taking advantage of the value associated with each key in the 2-3-4 tree. By making the value be the count of the number of times the key appears in the array, the correct number of duplicate values can be placed in the output array.

The `sortarray()` function should start by creating an empty 2-3-4 tree, loop through all the keys checking whether the key is already in the tree. If a key is already in the tree, increment its count and reinsert it in the tree with the correct count. Next, the function should traverse the tree in order to copy the keys back to the array, making any needed copies of duplicate keys. Show the input and output array contents, including some duplicate values.

An interesting “array” to sort is your source program. You can put each line of your source code into a Python array with an expression like

```
[line for line in open('my_source_code.py')]
```

A few blank lines in the source code will likely be duplicate values.

9.4 Modify the `Tree234.py` program to make a `Tree23` class so that it creates and works with 2-3 trees instead. It should display the tree and allow searches. It should also allow items to be inserted, but only if the parent of the leaf node (which is being split) does not also need to be split. In other words, it will allow insertions only at level 0 and level 1 of the tree and only when one or the other node on the insertion path is not full. It must handle the split cases for leaf nodes shown in [Figure 9-16](#). The next Programming Project explores how to insert items into deeper nodes. The `delete()` method is not required. If `insert()` is called on an existing key, the key’s value should be updated (and the node should not be split). Show how the tree grows as you add items. Try adding 10 different items and then searching the tree for both items it contains and items it does not contain to show its performance.

If you plan to solve both this Programming Project and the next one, it helps to set up the `insert()` method as follows. The `insert()` method should call a recursive method, `__insert()`, that takes a node object as a parameter, along with the key and value to insert. The `insert()` method calls `__insert()` on the root node and looks to see if a split happened on the root. If it does, `insert()` should make a new root node to hold the promoted item with the old root node and the split node as children.

To make this work, the `__insert()` method should return three items: a key, the key’s value, and a split node, all bundled as a Python tuple. When a split occurs, the key and value together are the item to promote, and the split is the new node that is the top of a subtree containing items having higher keys than the promoted item. If the key in the returned tuple is `None`, then no split occurred. The `__insert()` method is called recursively to descend the tree, but only one level down. If its node argument is `None`, that’s the base case of an empty tree (no root node), so it returns a simulated split with a tuple containing the item being inserted and `None` for the split node. The `insert()` method turns that into a new root node that is also a leaf node.

If the `__insert()` method gets a valid leaf node, it should determine where the new key would go, insert it, and determine whether that causes a split, returning a tuple of three `Nones` if no split is needed. If the `__insert()` method gets a valid internal node, it should check if either the internal node or the child where the insert should take place is not full and make the recursive call on the child. If the recursive call causes a split, the promoted item should be inserted in the internal node. If both nodes are full, it should raise an exception saying that kind of insertion is not allowed.

- 9.5 Extend the program in Programming Project 9.4 so that the `__insert()` routine is fully recursive and can handle situations with a full parent of a full child following the cases presented in [Figure 9-17](#). This allows insertion of an unlimited number of items. The base cases of `__insert()` are the same as before, but without the check that either the current node or the child where the insertion will take place is full, the split can continue propagating up the tree in the recursive calls. As before, show how the tree grows as you add items. Try adding 10 different items and then searching the tree for both items it contains and items it does not contain to show its performance.

10. AVL and Red-Black Trees

In This Chapter

- [Our Approach to the Discussion](#)
- [Balanced and Unbalanced Trees](#)
- [AVL Trees](#)
- [The Efficiency of AVL Trees](#)
- [Red-Black Trees](#)
- [Using the Red-Black Tree Visualization Tool](#)
- [Experimenting with the Visualization Tool](#)
- [Rotations in Red-Black Trees](#)
- [Inserting a New Node](#)
- [Deletion](#)
- [The Efficiency of Red-Black Trees](#)
- [2-3-4 Trees and Red-Black Trees](#)
- [Red-Black Tree Implementation](#)

As you learned in [Chapter 8](#), “[Binary Trees](#),” ordinary binary search trees offer important advantages as data storage structures. They enable quick search for an item with a given key, and quick insertion or deletion of an item. Other data storage structures, such as arrays, sorted arrays, linked lists, and sorted linked lists, perform one or the other of these activities slowly. Thus, binary search trees might appear to be the ideal data storage structure.

In [Chapter 9](#), “[2-3-4 Trees and External Storage](#),” you learned how storing multiple items per node and carefully controlling them during insertions could produce balanced trees. Maintaining balance in the trees keeps the time needed to find, insert, or delete an item consistent and quick. Unbalanced trees can be much slower by comparison. Ordinary binary search trees can become unbalanced depending on the order items are inserted.

This chapter explores ways to keep binary search trees balanced using AVL trees and red-black trees. Binary trees are simpler, in general, with a single item per node, and in some ways, easier to implement than multiway trees. They also are slightly easier to analyze because the number of items per node is a constant.

Surprisingly, there’s a direct correspondence between the multiway 2-3-4 tree and binary red-black tree, as you will learn.

Our Approach to the Discussion

We’re going to start with a simple approach to balancing binary trees: measure and correct. We add an extra field to every node to keep track of the height of the tree and keep updating it as items are inserted, moved, and removed. From that, it will be easy to see when the tree’s balance needs to be corrected. Later, we show a different approach that labels the nodes as either red or black. With just those two labels and some rules about how the labels are manipulated, it’s also possible to determine when the tree is balanced or not.

The **AVL tree** is the earliest kind of balanced tree. It’s named after its inventors: Adelson-Velskii and Landis. In AVL trees each node stores an additional piece of data: its height from the deepest leaf node in that subtree. This data is the easiest to understand conceptually and perhaps to implement too. The red-black tree is a bit more complicated conceptually, so we discuss how it works without showing an implementation. We also investigate its connection to the 2-3-4 tree of [Chapter 9](#).

Balanced and Unbalanced Trees

Let’s review how binary trees become unbalanced. When a group of keys is inserted in either ascending or descending order, the tree grows on only one side, like in the example shown in [Figure 10-1](#).



Figure 10-1 *Unbalanced tree resulting from items inserted in ascending order*

As mentioned in [Chapter 8](#), the nodes arrange themselves in a line with no forked branches. Because each node is larger than the previously inserted one, every node is a right child, so all the nodes are on one side of the root. The tree is maximally unbalanced. If you were to insert items in descending order, every node would be the left child of its parent, and the tree would be unbalanced on the other side. When the insertion order is only partly increasing or decreasing, every run of increasing or decreasing keys can make a linear string of nodes like this.

Degenerates to $O(N)$

When there are no branches, the tree becomes effectively a sorted linked list. The arrangement of data is one-dimensional instead of two-dimensional. Unfortunately, as with linked lists, you must now search through (on average) half the items to find the one with a particular key. In this **degenerate** situation, the speed of searching is reduced to $O(N)$, compared with the $O(\log N)$ of a balanced tree. Searching through 1,000,000 items in such an unbalanced tree would require an average of 500,000 comparisons, whereas for a balanced tree it requires only 20.

Data that's only partly sorted generates trees that are only partly unbalanced. If you use the Binary Search Tree Visualization tool from [Chapter 8](#) to attempt to generate trees with 31 nodes, some of them are more unbalanced than others. For example, the tree shown in [Figure 10-2](#) has 14 items, even though the request was to fill it with 31 items (the maximum that can be shown). The

Visualization tool prevents insertions below level 4, so the items that would have been inserted at level 5 or below were discarded.



Figure 10-2 *A partially unbalanced tree*

Although not as bad as a maximally unbalanced tree, this situation is not optimal for searching. Searching partially unbalanced trees takes time somewhere between $O(N)$ and $O(\log N)$, depending on how badly the tree is unbalanced.

Measuring Tree Balance

To know when a tree is unbalanced, we should have a way of measuring balance. With a metric, we can define some value or values as being “balanced,” and everything else can be considered unbalanced.

What should we measure? As mentioned in [Chapter 8](#), balanced trees should have an approximately equal number of nodes in their left and right sides. We could simply count the nodes on the left and the right. In [Figure 10-1](#) the root, node 44, has zero nodes to the left and three nodes to the right. That’s clearly unbalanced, but how about the root, node 49, in [Figure 10-3](#)? It has five nodes on the left and five nodes on the right. It’s more balanced but still has long sequences of nodes with only a single child.



Figure 10-3 *A symmetric binary tree*

There are two difficulties with simply counting the nodes to the left and the right of the root. The first is that you're looking only at the root. If you only measure the tree at the top, you could miss problems further down. It's better to apply the metric to *every subtree*. That way, you can achieve two things: you get a measure over the entire tree, and you can measure each subtree separately. Being able to measure subtrees allows you to note things like nodes 19 and 75 in [Figure 10-3](#) being fully balanced, whereas nodes 27 and 65 are quite unbalanced.

If you apply the node count technique to each subtree, you can certainly identify balanced and unbalanced nodes by subtracting the number of children on the right from the number on the left. Using those differences, zero means balanced, a positive value means there are more nodes on the left, and a negative value means there are more on the right.

To get a measure for the overall tree, you could try to combine the subtree measures into a single metric. If you were to add up all the left-right node count differences, you would get an overall metric, but that would still result in a value of zero for the tree in [Figure 10-3](#) because every node with a negative value has a matching positive value on the other side. That might be an interesting measure of symmetry, but it doesn't help decide when the tree is balanced to minimize searching.

Adding up the absolute values of the differences would be one way to measure the full tree's imbalance. That would total 14 for the tree in [Figure 10-3](#) (four each for nodes 27 and 65, three each for nodes 16 and 83, and zero for all the others because they are balanced). If you rearrange the nodes into the nearly

symmetric and balanced tree shown in [Figure 10-4](#), the overall metric goes down to 4 (one each for the nodes at level 2 with the keys 17, 21, 65, and 80).

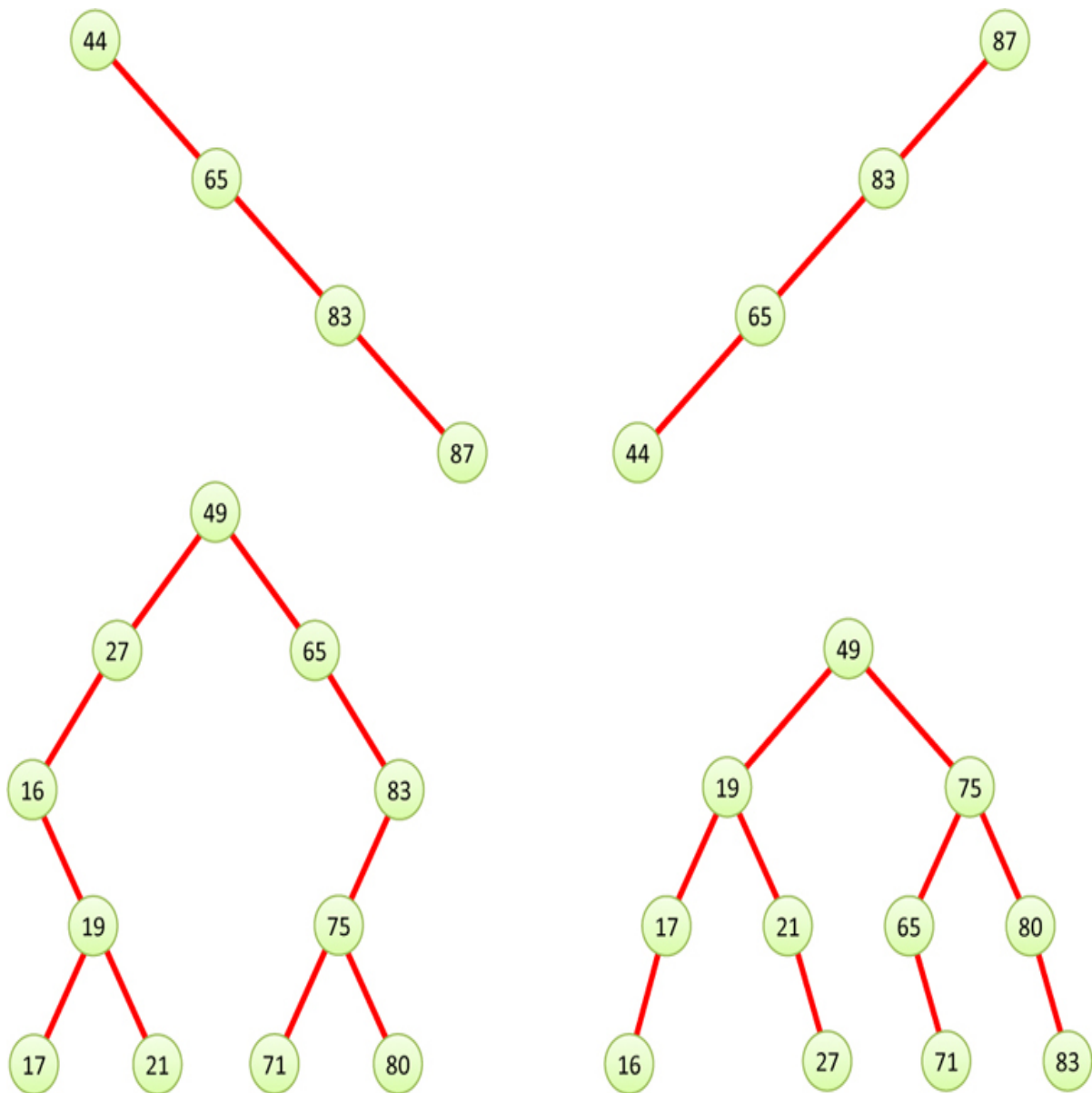


Figure 10-4 *A nearly symmetric, balanced binary tree*

That's definitely an improvement. The biggest impact to search performance, however, lies in the length of the path to the deepest leaf node. That becomes the longest path that a search must follow. In the degenerate case of insertion in ascending order of keys shown in [Figure 10-1](#), the longest path has length $N-1$. Counting nodes on both subtrees sort of captures the imbalance, but summing the absolute difference in node counts produces a metric of four for the tree in